

geocore — a shared Rust/WASM geospatial core for Dioxus apps (plan)

Mapping · urban · environmental — pure open-source Rust,
browser-first

Table of Contents

- [1. Purpose](#)
- [2. Guiding architecture](#)
 - [2.1. Style: layered "sans-IO" core with thin adapters \(hexagonal\)](#)
 - [2.2. Why not "one big crate"](#)
- [3. Workspace layout](#)
- [4. Crate-by-crate design](#)
 - [4.1. geocore-types — foundations](#)
 - [4.2. geocore-crs — projections](#)
 - [4.3. geocore-io — codecs \(bytes in, structs out; no I/O\)](#)
 - [4.4. geocore-index — spatial indexing & stores](#)
 - [4.5. geocore-ops — vector analysis, measurement, selection](#)
 - [4.6. geocore-raster — grids, band math, legends, hydrology](#)
 - [4.7. geocore-tiles — tile math & tiled sources](#)
 - [4.8. geocore-eo — remote-sensing domain logic](#)
 - [4.9. geocore-symbolology — cartography model](#)
 - [4.10. geocore-map — the renderer-agnostic map state machine \(the heart\)](#)
 - [4.11. geocore-render — renderer port](#)
 - [4.12. geocore-render-galileo — primary backend \(pure Rust\)](#)
 - [4.13. geocore-render-maplibre — interim backend \(JS interop, not JS logic\)](#)
 - [4.14. geocore-net / geocore-workers — platform adapters](#)
 - [4.15. geocore-ai — ML service clients \(typed, transport-agnostic\)](#)
 - [4.16. geocore-report — exports](#)
 - [4.17. geocore-dioxus — UI kit \(Dioxus 0.7\)](#)
 - [4.18. geocore-wasm — JS facade \(migration path\)](#)
- [5. Mapping the current canvas onto the crates](#)
- [6. WASM-in-the-browser engineering notes](#)
- [7. Testing & quality](#)
- [8. Delivery phases \(dependency order, no calendar estimates\)](#)
- [9. Key decisions & risks](#)
- [10. Appendix A — workspace Cargo.toml sketch](#)
- [11. Appendix B — core trait sketches](#)
- [12. Appendix C — naming note](#)

1. Purpose

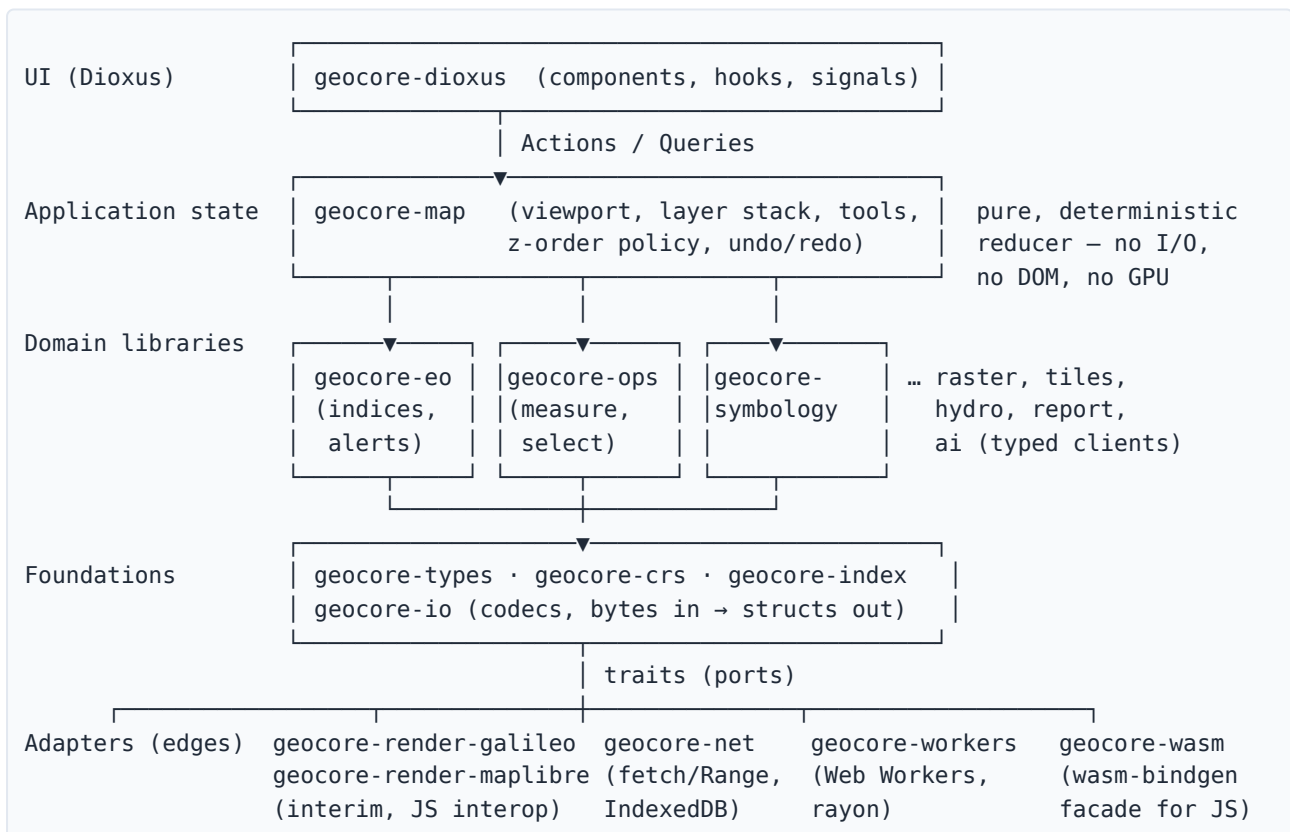
Several products (agriculture, urban, environmental) share the same map canvas behaviour that [SATELLITE_INTELLIGENCE_CANVAS_TOOLS.org](https://satellite-intelligence-canvas-tools.org) documents for the current React + Mapbox GL app: a layered canvas (basemaps → rasters → vectors → sketch chrome → tools), drawing / measurement / selection, a swipe comparator, remote-sensing indices with AOI clipping, hydrology, AI-service clients, symbology, import/export and reporting.

This plan describes a **Cargo workspace** — working name `geocore` (rename freely) — that factors that behaviour into reusable crates consumable by **Dioxus** front-ends, with these constraints:

1. Open-source Rust only; no JavaScript libraries as functional dependencies. The reference toolkits are the GeoRust ecosystem (`geo`, `geojson`, `geozero`, `geo-index`, `rstar`, `geotiff`, `kml`, `gpx`, `earcutr`, `geographiclib-rs`, ...), `geolibre-rust` (opengeos: pure-Rust/WASM whitebox tools, GeoTIFF/COG, PMTiles, projections, LiDAR, topology), `galileo` (wgpu map renderer), `proj4rs` / `geodesy` (pure-Rust CRS).
2. Browser-first: everything compiles to `wasm32-unknown-unknown`; heavy work runs client-side, as in the current project. Native (desktop / server) builds are a by-product, not a goal.
3. Renderer-agnostic core so the rendering backend can evolve (Galileo today, WebGPU later) without touching domain logic.

2. Guiding architecture

2.1. Style: layered "sans-IO" core with thin adapters (hexagonal)



Rules that keep the core portable:

- **Sans-IO:** domain crates never perform network, file or DOM access. They expose "what bytes/ranges do you need" state machines (the pattern `geolibre-pmtiles` uses for its `Extractor`) and accept bytes back. Adapters do the fetching.
- **Sans-render:** `geocore-map` describes the desired scene (layers, styles, order, camera) as data; a `MapRenderer` trait implementation applies diffs.
- **Deterministic state:** one `MapState` + `Action` enum reducer, replayable and unit-testable natively without a browser. `Dioxus` signals wrap it.
- **Feature-gated backends** (`web`, `native`, `galileo`, `maplibre`, `whitebox`, `threads`) so each app pulls only what it ships.
- **Single-threaded by default on wasm**, optional `threads` feature (`wasm-bindgen-rayon`, requires COOP/COEP headers); heavy raster work goes to Web Workers via a typed message protocol (`postcard`) rather than shared memory.

2.2. Why not "one big crate"

The current 29k-line page shows what happens when everything lives together. A workspace with small crates gives: independent compile units (wasm build times), per-app feature selection (binary size), clear ownership, and the ability to publish foundations (`types/crs/io`) separately from opinionated domain logic (`eo/alerts`).

3. Workspace layout

```
geocore/
├─ Cargo.toml                # [workspace] + shared [workspace.dependencies] / lints /
profiles
├─ rust-toolchain.toml       # stable; wasm32-unknown-unknown target
├─ .cargo/config.toml        # wasm rustflags (+simd128, opt-level, getrandom js)
├─ crates/
│   ├─ geocore-types          # domain model, ids, errors, units
│   ├─ geocore-crs            # CRS registry + transforms (proj4rs / geodesy)
│   ├─ geocore-io             # codecs: vector/raster/table formats, sans-I0
│   ├─ geocore-index          # spatial indexes, feature stores, viewport caches
│   ├─ geocore-ops            # vector analysis, measurement, selection queries
│   ├─ geocore-raster         # grid model, band math, legends, contours, hydrology
│   └─ geocore-tiles          # tile math, WMS/WMTS/XYZ URL builders, PMTiles, MVT, STAC,
Terrain-RGB
│   └─ geocore-eo             # remote-sensing domain: index catalog, evalscripts,
scenes, time series, alert engines
│   └─ geocore-symbolology    # style model, classification, color ramps, labels, popups,
legend specs
│   └─ geocore-map            # renderer-agnostic map state machine (layers, z-order,
tools, swipe, timeline)
│   └─ geocore-ai             # typed clients for ML services (SAM, SegFormer, trees,
fields, crop, flood, GeoAI chat)
│   └─ geocore-report         # exports: GeoJSON/SHP/KML/CSV/XLSX/GeoTIFF/DOCX/PDF/PNG
│   └─ geocore-net            # HTTP (Range), caching (IndexedDB / memory), auth tokens –
adapter
│   └─ geocore-workers        # Web Worker / rayon job runner for raster & hydrology –
adapter
│   └─ geocore-render         # `MapRenderer` port + shared scene-diff logic
│   └─ geocore-render-galileo # Galileo (wgpu → WebGL/WebGPU) backend
│   └─ geocore-render-maplibre # interim MapLibre GL JS backend via wasm-bindgen
(globe/terrain today)
│   └─ geocore-dioxus         # Dioxus 0.7 components + hooks
│   └─ geocore-wasm           # wasm-bindgen facade (lets the existing React app adopt
the core incrementally)
├─ apps/
│   ├─ demo-dioxus            # reference app: all tools wired, used for e2e
│   └─ bench-native           # criterion benches for ops/raster
├─ fixtures/                  # small GeoJSON / GeoTIFF / SHP / PMTiles test data
└─ xtask/                     # build/test tasks (wasm-pack, wasm-opt, size report)
```

4. Crate-by-crate design

4.1. geocore-types — foundations

- Geometry: re-export `geo-types` (`Point`, `LineString`, `Polygon`, `Multi*`, `Geometry`) wrapped in a `Geom<C: Crs>` new-type or a runtime `CrsTagged { geom, srid }` so mixing CRSs is a compile/runtime error, never a silent bug.
- `Feature`, `FeatureCollection` (typed properties via `serde_json::Value` or a `PropertyMap` with typed accessors), stable `FeatureKey` (the current `computeStableGisFeatureKey` behaviour).
- `Layer` descriptor (`id`, `name`, `kind: Vector | Raster | Tiles | Image | Analysis`, `source` `provenance Upload | Api | ArcGis | Stac | Tool`, `visibility`, `opacity`, `order tier`).
- `Aoi` (`Drawn | FromLayer { layer, selected_only, filter } | Viewport`), `Bbox`, `LonLat`.

- Raster descriptor: `GridSpec` (size, geo-transform, CRS, nodata), `Band` dtype enum.
- Time: `SceneDate`, `DateRange`, `TimeWindow` (use `time crate`; `web-time` on `wasm`).
- Units: `Length`, `Area` with unit enums (`m/km/ft/mi`, `m²/ha/acre/km²`).
- Errors: one `GeoError` per crate via `thiserror`; `no_std`-friendly where feasible.
- Traits (ports): `TileSource`, `ByteRangeSource`, `Clock`, `Rng`.

4.2. geocore-crs — projections

- Backend: `proj4rs` (pure Rust, `wasm32` target, `proj4js`-compatible API, EPSG via `crs-definitions`); optional `geodesy` for pipeline / datum-shift work.
- API: `Crs::epsg(4326)`, `Transform::new(from, to)`, `transform_geometry`, `transform_bbox` (densified edges), Web-Mercator helpers (`lonlat_to_mercator`, `mercator_bounds_for_tile`).
- WKT/PRJ ingestion for shapefiles: `proj4wkt` (3liz) → proj string.
- Geodesic distance/area: `geographiclib-rs` (Karney) with `geo` Haversine fallback.

4.3. geocore-io — codecs (bytes in, structs out; no I/O)

Format	Read	Write	Crates
GeoJSON / TopoJSON	✓	✓	<code>geojson</code> , own <code>TopoJSON→GeoJSON</code> arc decoder
Shapefile (.zip of shp/shx/dbf/prj/cpg)	✓	✓	<code>shapefile</code> , <code>zip</code> (deflate only, <code>wasm-safe</code>), <code>encoding_rs</code>
KML / KMZ	✓	✓	<code>kml</code> , <code>zip</code>
GPX	✓	–	<code>gpx</code>
WKT / WKB / MVT / CSV / FlatGeobuf	✓	✓	<code>geozero</code> (+ <code>flatgeobuf</code>)
CSV / XLSX tables → points	✓	✓	<code>csv</code> , <code>calamine</code> (read), <code>rust_xlsxwriter</code> (write)
GeoTIFF / COG (incl. Terrain-RGB PNG)	✓	✓	<code>geotiff</code> / <code>tiff</code> , or <code>geolibre-wasm</code> <code>GeoTiffReader</code> / <code>CogBuilder</code> behind <code>feature = "geolibre"</code> ; image for PNG/JPEG/WebP + world files (<code>.tifw/.pgw/.jgw/.wld</code>)
LAS / LAZ (optional)	✓	–	<code>las</code> (+ <code>laz</code>)
IFC / BIM footprint (optional)	✓	–	<code>ifc-rs</code> or defer; keep as an extension trait

Design: `trait Codec { fn sniff(bytes) -> Option<Format>; fn decode(bytes, opts) -> Dataset }; Dataset = Vector<FeatureCollection> | Raster(RasterImage) | Table(Rows)`. Import metadata (geometry kind detection, CRS guess, feature count) mirrors `siCustomLayerFactory`.

4.4. geocore-index — spatial indexing & stores

- `geo-index` (`Flatbush` / `KDBush`, zero-copy, packed) for static layers; `rstar` for dynamic edits.

- `FeatureStore` (arena of features + index + attribute columns) with viewport queries and a `ViewportFeatureCache` for streamed ArcGIS/WFS layers.
- Attribute query engine for "definition query" / "select by attributes" (small expression parser → predicate).

4.5. geocore-ops — vector analysis, measurement, selection

- `geo` algorithms: area, length, centroid, convex hull, simplify (RDP/VW), buffer, boolean ops (union/intersection/difference/clip), contains/intersects/within, `earcutr` triangulation.
- Measurement engine (ports the 12 modes): distance, area, perimeter, rectangle, circle, radius, bearing/azimuth, angle, coordinate, elevation, 3D distance, height — geodesic by default, unit conversion via `geocore-types` units.
- Selection queries: point (with overlap stack), rectangle, polygon, lasso, circle, line, trace; set modes New/Add/Remove/Subset; "select by location" relations (intersects, within, contains, overlaps, touches, within-distance, completely-contains).
- Sketch geometry helpers: circle from centre+edge with cardinal handles, rectangle from two corners, polygon ring closing, vertex insert/move/delete.

4.6. geocore-raster — grids, band math, legends, hydrology

- `Raster<T>` over `ndarray` (bands × rows × cols) + `GridSpec`; typed dtypes; nodata mask.
- Band-math expression engine: parse `"(B8-B4)/(B8+B4)"` → AST → vectorised evaluation (also emits Sentinel Hub evalscript JS **text** for server-side WMS — generating text is not "using JavaScript").
- Reclass, thresholds, histograms, percentile stretch, class-area statistics (the Layer-Live legend analyze), zonal statistics over `Aoi` (rasterise with `geo-rasterize`).
- Colour ramps & legends: `colourous` ramps + own 10-class agri ramps; `Legend` spec output.
- Contours: `contour` crate (marching squares) and isobands.
- Hydrology & terrain: slope, aspect, hillshade, TPI/TRI/TWI, D8 flow direction, flow accumulation, Strahler/Shreve streams, watershed/basins. Two implementations behind features: `native-hydro` (own small D8 kernels, good for AOI-scale DEMs) and `whitebox` (delegate to `whitebox-wasm` / `geolibre-tools` `wbcore::Tool` registry — hundreds of vetted tools, runs in a worker).
- Terrain-RGB decode/encode (Esri/Mapbox formula) for the DEM proxy tiles.
- Raster georeferencing: GCP affine / polynomial solve (`nalgebra`), world-file emit.

4.7. geocore-tiles — tile math & tiled sources

- XYZ/TMS/quadkey math, WebMercator bounds, max-native-zoom per provider (port of `rasterTileZoom`).
- URL builders: XYZ templates, WMS GetMap (incl. `GEOMETRY` WKT clip, `EVALSCRIPT` base64, `TIME`, `MAXCC`), WMTS, ArcGIS MapServer/ImageServer export tiles.
- `PMTiles`: `geolibre-pmtiles` (sans-IO extractor + tile math) or `pmtiles` crate; MVT decode via `geozero` (`mvt` feature) — replaces the current hand-rolled `@mapbox/vector-tile` + `pbf` FTW loader.

- STAC: `stac` crate types + sans-IO search request builder / response parser (Planetary Computer, CDSE).
- Basemap catalog as data (the 21 entries from the report) with attribution and fallback rules (401/403 → Esri World Imagery).

4.8. geocore-eo — remote-sensing domain logic

- Index catalog as data: core S2 indices, agro composites (+Δ), mangrove, per-collection (S3/S5P/S6/CCM/DEM/CLMS), ASTER, CHIRPS analytics — each with formula AST, bands, default ramp, legend classes, time window (30/45/60/90/120 d) and provider profile.
- Provider registry (Sentinel Hub, CDSE, Landsat/HLS, ASTER, commercial VHR placeholders) with layer-profile filtering.
- Scene resolution: latest / nearest / previous scene from STAC results with cloud filter and cache policy (pure functions over STAC items).
- AOI clip planning: chunking of a multipolygon into WMS `GEOMETRY` strings under URL budget (port of `siSentinelAoiWmsStack` / `sentinelHubWmsAoiClip`), producing `ClippedTileStack { chunks: [(bounds, url)] }` consumed by `geocore-map`.
- Time-series analytics (zonal series, smoothing, anomalies, phenology metrics), LULC composition.
- Rule engines: crop alerts (ΔCHAS), WAPI/ISS irrigation stress, stress zones, flood thresholding (SAR dB) — pure, testable with fixtures.

4.9. geocore-symbology — cartography model

- `SymbologyStyle` (single, unique, graduated colour, size, colour+size, dot density, threshold markers), `ClassMethod` (Jenks natural breaks, quantile, equal interval), `ColorRamp` (viridis/blues/greens/plasma/magma/turbo + custom), stroke/fill styles, point symbol gallery.
- Label style (field, font stack, halo, min/max zoom), popup config, legend spec.
- ArcGIS `drawingInfo` renderer → `SymbologyStyle` converter (keep ArcGIS Online looks).
- Converters: `SymbologyStyle` → Galileo style; → MapLibre style JSON (for the interim backend and for exporting styles).

4.10. geocore-map — the renderer-agnostic map state machine (the heart)

- `MapState`: camera (`Viewport: lon/lat/zoom/bearing/pitch`, 2D/3D flag, globe flag), `LayerStack`, active `Aoi`, `SelectionState`, `DrawState`, `MeasureState`, `SwipeState`, `Timeline`, `IdentifyState`, undo/redo history.
- `Action` enum (`SetBasemap`, `AddLayer`, `ReorderLayer`, `ToggleVisibility`, `SetOpacity`, `StartDraw(tool)`, `AddVertex`, `CommitSketch`, `MeasureMode(..)`, `Select { tool, set_mode, geometry }`, `SwipeSet { mode, before, after, split }`, `TimelineStep`, ...) → `fn reduce(&mut MapState, Action) -> Vec<Effect>`.
- `Effect` = things adapters must do (fetch tiles, run worker job, emit renderer diff).
- **Z-order policy** as an explicit tier table that reproduces Part I of the report: `Terrain` → `Basemap` → `AoiFillUnderRaster` → `AnalysisRaster` → `ImportedVector` → `ToolResultVector` → `AoiOutline` → `SelectionHighlight` → `SketchChrome`; every layer declares its tier; the policy is enforced in one place (no more scattered `moveLayer` calls).

- Flicker-free raster swap: the ping-pong double-source strategy modelled as a `RasterSlotPair` state machine (load hidden slot → swap opacity on ready).
- Swipe: `SwipeState { mode: Dates | Layers | Both, before, after, split }` producing two layer descriptors; renderer adapters implement the clip (Galileo: scissor rect / second view; MapLibre interim: second map + CSS clip-path).
- Interaction adapters: pointer events normalised (`PointerAction`) so Dioxus and tests feed the same reducer.

4.11. geocore-render — renderer port

```
pub trait MapRenderer {
    fn mount(&mut self, canvas: CanvasHandle, opts: RendererOptions) -> Result<(),
    RenderError>;
    fn apply(&mut self, diff: SceneDiff);           // add/remove/reorder layers, paint
    updates
    fn set_camera(&mut self, cam: &Camera, anim: Animation);
    fn project(&self, lonlat: LonLat) -> ScreenPt;    // for HTML markers / popups
    fn unproject(&self, pt: ScreenPt) -> LonLat;
    fn query_features(&self, pt: ScreenPt, radius_px: f32) -> Vec<FeatureHit>;
    fn snapshot_png(&self) -> Result<Vec<u8>, RenderError>;
}
```

`SceneDiff` is computed by `geocore-map` from two `LayerStack` snapshots (like React's reconciliation but explicit), so backends stay thin.

4.12. geocore-render-galileo — primary backend (pure Rust)

- `galileo` 0.2.x: raster tile layers, vector tile layers with styling, feature layers, 3D view/objects, `wgpu` → `WebGL2/WebGPU`; roadmap v0.3 adds globe/terrain.
- Mapping: `Basemap` → `RasterTileLayer`; imported vectors → `FeatureLayer` with `geocore-symbology` styles; analysis rasters/images → image layers; `PMTiles/MVT` → vector tile layer; markers/popups → DOM elements positioned via `project()`.
- Gaps to track: globe projection & terrain (interim via MapLibre backend), label collision, heatmap/cluster styles (roadmap v0.4).

4.13. geocore-render-maplibre — interim backend (JS interop, not JS logic)

- Thin `wasm-bindgen` bindings to MapLibre GL JS **only as a rendering device** (globe, terrain, hillshade, labels) while Galileo matures. All logic stays in Rust; the JS surface is a handful of `addSource/addLayer/moveLayer/setPaintProperty` calls. Feature-gated; deletable later without touching domain crates.

4.14. geocore-net / geocore-workers — platform adapters

- HTTP: `gloo-net` on web (`fetch`, Range headers for COG/PMTiles), `request` on native; token providers (Sentinel Hub, ArcGIS portal, API keys) as a `Credentials` trait.
- Cache: in-memory LRU (`lru`), IndexedDB (`idb` / `gloo-storage`) for tiles, scene lists, saved maps, user prefs (replaces `localStorage` sprawl).

- Workers: `gloo-worker` typed bridge; job kinds: decode raster, band math, hydrology, contour, shapefile parse, report build. Optional `wasm-bindgen-rayon` when the host can set COOP/COEP.

4.15. geocore-ai — ML service clients (typed, transport-agnostic)

- Request/response types + endpoint builders for the existing Python services: SAM, SAM2 refine, SegFormer, temporal transformer, tree detection, field boundary (AFD/FTW/Delineate-Anything, SEN2SR), image classification, GeoAI chat; plus LLM provider adapters (Gemini, Claude, DeepSeek, Ollama) as plain HTTP JSON.
- Post-processing that belongs client-side: mask PNG → polygons (`geocore-raster` vectorise), clip results to AOI, class styling, export packs.
- Optional on-device inference behind `feature = "ondevice"` via `tract` or `candle` (ONNX in `wasm`) for small models (e.g. tree crowns fallback).

4.16. geocore-report — exports

- Vector: GeoJSON, Shapefile zip, KML/KMZ, CSV, XLSX (`rust_xlsxwriter`), GPKG (via `geozero` WKB + `rusqlite` — native only; `wasm` via `sql.js`-free `sqlite-wasm-rs` is optional).
- Raster: GeoTIFF (Float32 index + RGB + world files), PNG snapshots.
- Documents: DOCX (`docx-rs`), PDF (`printpdf` / `genpdf`), charts rendered with `plotters` (canvas backend on web, bitmap on native) — mirrors the 14 time-series export kinds.

4.17. geocore-dioxus — UI kit (Dioxus 0.7)

- `MapView` component: mounts a `<canvas>` via `onmounted`, owns the renderer, wires pointer/keyboard events to `Action=s`; exposes `=use_map()` (signals for `MapState` slices) and `use_map_effects()` (drives `geocore-net` / `geocore-workers`).
- Components mirroring the current chrome: `LayerList` (TOC with tier-aware ordering), `BasemapGallery`, `DrawToolbar`, `MeasurePanel`, `SelectionWorkbench`, `SwipeControl`, `TimelineBar`, `LegendPanel`, `GoToXy`, `SearchBox`, `SymbolologyStudio`, `LabelingPanel`, `PopupConfigurator`, `FeaturePopup` (positioned via `project()`), `RemoteSensingPanel`, `HydroPanel`, `AlertCenter`, `ReportExportMenu`.
- Headless first: each component takes state + emits `Action`; styling via CSS classes (Tailwind-compatible) so apps can re-skin.

4.18. geocore-wasm — JS facade (migration path)

- `wasm-bindgen` exports for the existing React app: codecs, CRS, measurement, index catalog, AOI clip planning, z-order policy. Lets the current product adopt the Rust core incrementally while the Dioxus apps are built.

5. Mapping the current canvas onto the crates

Report item (SI canvas)	geocore home
Engine / globe / fog / sky	<code>geocore-render-*</code> (globe: MapLibre interim → Galileo v0.3); sky survey optional layer

Report item (SI canvas)	geocore home
Terrain DEM proxy, hillshade, exaggeration	geocore-raster (Terrain-RGB), geocore-render-*
Basemap catalog (21), fallback, max-native-zoom	geocore-tiles (data) + geocore-map (SetBasemap)
AOI fills under rasters / outlines above / draft on top	geocore-map z-order tiers
Sentinel WMS drawn-AOI & Layers-AOI stacks, ping-pong	geocore-eo (clip planning) + geocore-map (RasterSlotPair)
Crop-AI / STAC thumb / CHIRPS / DEM / hydro / flood / SegFormer image rasters	geocore-raster + image layers via SceneDiff
Imported vectors, ArcGIS drawingInfo, labels	geocore-io, geocore-symbolology, geocore-index
FTW Global PMTiles	geocore-tiles (PMTiles + MVT)
Tool-result vectors (SAM, trees, fields, streams...)	geocore-ai post-processing → layers
Selection highlight, identify popups	geocore-ops + geocore-dioxus FeaturePopup
Draw / measure chrome & HUD	geocore-ops + geocore-map state + geocore-dioxus
HTML markers (alerts, pins)	geocore-dioxus positioned by project()
Swipe tool (dates / layers / both)	geocore-map SwipeState + backend clip
Right-rail toolbox, processing portal, timeline	geocore-dioxus shells
Remote-sensing indices, providers, scene dates, legends	geocore-eo, geocore-raster
Hydro watershed, well-site, MCDA	geocore-raster (native-hydro / whitebox), geocore-report
Weather (Open-Meteo), time series, 14 export kinds	geocore-eo (analytics), geocore-net, geocore-report
GeoAI assistants	geocore-ai (LLM adapters) + map command executor in geocore-map
Import formats (SHP/KML/GPX/CSV/XLSX/GeoTIFF/IFC)	geocore-io
Saved maps, prefs, snapshots	geocore-net cache + serialisable MapState

6. WASM-in-the-browser engineering notes

- Target `wasm32-unknown-unknown`; build with `dx` (Dioxus CLI) for apps, `wasm-pack` for `geocore-wasm`; post-process with `wasm-opt -O3`; enable `+simd128` for raster kernels.
- Async: `wasm-bindgen-futures + futures`; no `tokio` runtime in core crates (use `async-trait-free impl Future` APIs; adapters pick the executor).
- Time & randomness: `web-time`, `getrandom` with `js` feature; `fastrand` for non-crypto.
- Memory: keep rasters in typed `Vec<f32>=>u8`; move big buffers to workers with `Transferable ArrayBuffer`; cap textures (8192 px) as today.
- Binary size: split heavy optional tools (`whitebox`, `las`, `docx`, `printpdf`) behind features and, for the demo app, load them as a **separate worker wasm** so the main bundle stays small.
- Interop boundary: only `geocore-render-maplibre` and `geocore-dioxus` touch `web-sys=>js-sys`; everything else must compile with `--target wasm32` **and** natively.

- Threads: optional `threads` feature (`wasm-bindgen-rayon`); requires `Cross-Origin-Opener-Policy: same-origin` and `Cross-Origin-Embedder-Policy: require-corp`.
- Storage: IndexedDB for tile/scene caches and saved maps; OPFS for large local rasters when available.

7. Testing & quality

- Core crates: `cargo test` natively (fast), `proptest` for geometry/measurement, golden fixtures (GeoJSON/SHP/GeoTIFF/PMTiles) in `fixtures/`.
- `wasm`: `wasm-bindgen-test` in headless Chrome/Firefox for `geocore-io`, `geocore-net`, renderer smoke tests; Playwright-driven e2e on `apps/demo-dioxus`.
- Reducer property: replaying any `Action` log yields identical `MapState` (snapshot tests) — this is what makes the tool logic reusable across products.
- Benchmarks: `criterion` for band math, hydrology, index build; size report per crate in CI (`twiggy`).
- Lints: `clippy -D warnings`, `cargo deny` (licenses: MIT/Apache/BSD only), `typos`.

8. Delivery phases (dependency order, no calendar estimates)

1. **Skeleton** — workspace, CI (native + `wasm32`), `geocore-types`, `geocore-crs`, `geocore-index`; basemap catalog data; `MapState` reducer with camera + layer stack + z-order tiers; Galileo backend showing raster basemaps in the demo app.
2. **Vector core** — `geocore-io` (GeoJSON, SHP zip, KML/KMZ, GPX, CSV/XLSX), imported layer rendering with `geocore-symbolology`, TOC, draw/edit, measurement, selection workbench, identify popups, vector exports.
3. **Raster & EO** — `geocore-raster` grids, band math, legends, class areas; `geocore-tiles` WMS builders + AOI clip planning; ping-pong raster slots; scene date resolution via STAC; Layer-Live legend; **swipe tool**; GeoTIFF import/export; PMTiles/MVT.
4. **Terrain & hydrology** — Terrain-RGB, hillshade, 2D/3D camera (MapLibre interim backend for globe/terrain), contours, D8 hydrology (`native-hydro`), `whitebox` worker path, well-site / MCDA overlays.
5. **Intelligence** — `geocore-eo` alert engines and time-series analytics, `geocore-ai` service clients + mask vectorisation, weather (Open-Meteo) client, `geocore-report` DOCX/PDF/XLSX.
6. **UI kit & migration** — polish `geocore-dioxus` components, `geocore-wasm` facade for the React app, documentation site (mdBook) with per-crate examples.

9. Key decisions & risks

Decision	Choice	Risk / mitigation
Renderer	Galileo primary; MapLibre interim behind a trait	Galileo lacks globe/terrain/labels today; the <code>MapRenderer</code> port isolates it; revisit when v0.3 lands

Decision	Choice	Risk / mitigation
Projections	proj4rs (+ geodesy optional)	Not full PROJ; sufficient for EPSG:4326/3857/UTM/LCC; WKT via proj4wkt
Hydrology	own D8 kernels + optional whitebox-wasm	whitebox wasm is large → worker + lazy load
GeoTIFF	geotiff≠tiff vs geolibre-wasm reader	choose per feature; COG streaming needs Range → sans-IO reader
Formats without mature Rust	IFC, GPKG in browser, LAZ	keep behind optional features / native-only; document gaps
Threads in browser	opt-in only	COOP/COEP hosting requirement
State model	single reducer	discipline needed to keep effects out of the core; enforce with <code>#![forbid(unsafe_code)]</code> and no <code>web-sys</code> dep in core crates

10. Appendix A — workspace Cargo.toml sketch

Version numbers below are indicative only; resolve them with `cargo add` at implementation time and verify each crate builds for `wasm32-unknown-unknown`.

```

[workspace]
resolver = "2"
members = ["crates/*", "apps/*", "xtask"]

[workspace.package]
edition = "2021"
license = "MIT OR Apache-2.0"
rust-version = "1.85"

[workspace.dependencies]
# foundations
geo = "0.30"
geo-types = "0.7"
geojson = "0.24"
geozero = { version = "0.15", default-features = false, features = ["with-geo", "with-geojson", "with-wkt", "with-wkb", "with-mvt", "with-csv"] }
geo-index = "0.2"
rstar = "0.12"
proj4rs = { version = "0.1", default-features = false, features = ["geo-types"] }
crs-definitions = "0.4"
geographiclib-rs = "0.2"
earcutr = "0.4"
ndarray = "0.16"
contour = "0.13"
geo-rasterize = "0.1"
colourous = "1"
# formats
shapefile = "0.6"
kml = "0.8"
gpx = "0.10"
geotiff = "0.1"
tiff = "0.9"
image = { version = "0.25", default-features = false, features = ["png", "jpeg", "webp"] }
zip = { version = "2", default-features = false, features = ["deflate"] }
csv = "1"
calamine = "0.26"
rust_xlsxwriter = "0.80"
docx-rs = "0.4"
printpdf = "0.7"
plotters = { version = "0.3", default-features = false, features = ["line_series", "bitmap_backend"] }
pmtiles = "0.12" # or geolibre-pmtiles
stac = "0.12"
# runtime / platform
serde = { version = "1", features = ["derive"] }
serde_json = "1"
postcard = "1"
thiserror = "2"
time = { version = "0.3", features = ["serde", "parsing", "formatting"] }
web-time = "1"
futures = "0.3"
wasm-bindgen = "0.2"
wasm-bindgen-futures = "0.4"
js-sys = "0.3"
web-sys = "0.3"
gloo-net = "0.6"
gloo-worker = "0.5"
idb = "0.6"
dioxus = { version = "0.7", features = ["web"] }
galileo = { version = "0.2", default-features = false, features = ["wgpu", "geojson",

```

```

"image"] }

[profile.release]
opt-level = "z"
lto = "fat"
codegen-units = 1
panic = "abort"
strip = true

[profile.release.package."*"]
opt-level = 3 # keep numeric kernels fast; app crates stay size-optimised

```

11. Appendix B — core trait sketches

```

/// Sans-I/O byte-range consumer (COG, PMTiles, FlatGeobuf).
pub trait RangeReader {
    fn wanted(&self) -> Vec<ByteRange>;
    fn feed(&mut self, offset: u64, bytes: &[u8]) -> Result<(), IoError>;
    fn is_done(&self) -> bool;
}

/// A tiled source the map can draw (XYZ, WMS-as-tiles, PMTiles, in-memory image pyramid).
pub trait TileSource {
    fn scheme(&self) -> TileScheme; // XYZ / TMS / WMS / PMTiles
    fn url_for(&self, z: u8, x: u32, y: u32) -> Option<String>;
    fn bounds(&self) -> Option<Bbox>;
    fn max_native_zoom(&self) -> Option<u8>;
}

/// Everything a UI can do to the map; the only mutation path.
pub enum Action {
    Camera(CameraAction),
    Basemap(BasemapId),
    Layer(LayerAction), // add / remove / reorder / visibility / opacity / style
    Aoi(AoiAction), // set drawn / from-layer / clear
    Draw(DrawAction), // start(tool) / vertex / undo / commit / cancel
    Measure(MeasureAction),
    Select(SelectAction),
    Swipe(SwipeAction),
    Timeline(TimelineAction),
    Identify(IdentifyAction),
}

/// Side-effects the reducer asks adapters to perform.
pub enum Effect {
    RenderDiff(SceneDiff),
    Fetch { req: HttpRequest, on_done: EffectId },
    RunJob { job: WorkerJob, on_done: EffectId },
    Persist { key: String, bytes: Vec<u8> },
}

pub fn reduce(state: &mut MapState, action: Action) -> Vec<Effect>;

```

12. Appendix C — naming note

The request mentioned "geoliber-rust"; this plan assumes **geolibre-rust** (`opengeos/geolibre-rust`), the pure-Rust/WASM toolkit (whitebox tools, GeoTIFF/COG, PMTiles, projections, LiDAR, topology). Its `sans-I0` extractor design is adopted here as the pattern for all remote-range readers.